

Relatório da Solução

Objetivo da solução

Desenvolver uma aplicação em Python capaz de gerar automaticamente um catálogo digital e uma galeria de imagens relacionadas ao município de Alegrete-RS.

A solução tem como finalidade produzir artefatos reutilizáveis (HTML, JSON e imagens organizadas) que possam contribuir para o enriquecimento do portal alegrete.org.

LLM(s) utilizada(s)

Foi utilizada a ferramenta ChatGPT (OpenAI) como apoio no desenvolvimento do projeto.

A LLM foi utilizada para:

- estruturação inicial do código;
- sugestões de melhorias;
- correção de erros;
- otimização da organização do projeto;
- apoio na documentação.

Todas as implementações foram testadas, validadas e ajustadas manualmente pelo estudante.

Principais dificuldades encontradas

- ausência inicial do arquivo CSV;
- tratamento de imagens externas;
- validação de dados inconsistentes;
- execução em diferentes ambientes (local e Colab);
- organização da geração automática do HTML.

Resultados obtidos

A solução final gerou com sucesso:

- catálogo em HTML interativo;
- base de dados em JSON;
- download e organização de imagens;

- relatório automático de execução;
- estrutura de dados reutilizável para o portal.

O sistema executou corretamente e permitiu atualização simples dos conteúdos via CSV.

Possíveis evoluções futuras

- integração com banco de dados (SQLite/PostgreSQL);
- publicação automática no portal alegrete.org;
- inclusão de mapas interativos (geolocalização);
- interface web para cadastro de conteúdos;
- suporte a vídeos e multimídia.

CÓDIGO PYTHON (ENTREGUE EM ARQUIVO [.py](#))

[galeria.py](#)

```
from pathlib import Path
from dataclasses import dataclass
from html import escape
from datetime import datetime
from collections import Counter
import json

import pandas as pd
import requests

# =====
# CONFIGURAÇÃO
# =====

@dataclass
class Config:
    csv: str = "fotos_alegrete.csv"
    html: str = "portal_alegrete.html"
    json_file: str = "catalogo.json"
    relatorio: str = "relatorio_execucao.txt"
    imagens: str = "imagens"
    timeout: int = 10
```

```
CFG = Config()
Path(CFG.imagens).mkdir(exist_ok=True)
```

```
COLUNAS = [
    "titulo",
    "categoria",
    "imagem",
    "legenda",
    "localizacao",
    "fonte"
]
```

```
# =====
# CSV
# =====
```

```
def criar_csv_exemplo():
    dados = [
        {
            "titulo": "Praça Getúlio Vargas",
            "categoria": "Turismo",
            "imagem": "https://picsum.photos/900/500?1",
            "legenda": "Ponto central de Alegrete",
            "localizacao": "Alegrete-RS",
            "fonte": "https://picsum.photos"
        }
    ]

    pd.DataFrame(dados).to_csv(
        CFG.csv,
        index=False,
        encoding="utf-8"
    )
```

```
def carregar_dados():
    arquivo = Path(CFG.csv)

    if not arquivo.exists():
        criar_csv_exemplo()
        print("CSV criado automaticamente.")

    df = pd.read_csv(arquivo)

    faltando = set(COLUNAS) - set(df.columns)
```

```

if faltando:
    raise ValueError(f"CSV inválido. Colunas faltando: {faltando}")

return df

# =====
# IMAGENS (ROBUSTO)
# =====

def baixar_imagem(sessao, url, nome):
    destino = Path(CFG.imagens) / f"{nome}.jpg"

    # cache local
    if destino.exists():
        return str(destino)

    # arquivo local direto
    if Path(url).exists():
        return url

    try:
        r = sessao.get(url, timeout=CFG.timeout)
        r.raise_for_status()

        if "image" not in r.headers.get("Content-Type", ""):
            return url

        destino.write_bytes(r.content)
        return str(destino)

    except Exception:
        return url

# =====
# HTML
# =====

def criar_card(item):
    return f"""
<div class="card" data-cat="{escape(item['categoria'])}">
    

    <div class="conteudo">
        <h3>{escape(item['titulo'])}</h3>

```

```

        <p>{escape(item['legenda'])}</p>
        <p> 📍 {escape(item['localizacao'])}</p>
        <small>{escape(item['categoria'])}</small>
    </div>
</div>

```

```

def gerar_html(cards, categorias, total):
    data = datetime.now().strftime("%d/%m/%Y %H:%M")

    options = "".join(f"<option>{c}</option>" for c in categorias)

    return f"""
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8">
<title>Catálogo Alegrete</title>

<style>
body {{
    font-family: Arial;
    padding: 30px;
}}

.grid {{
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(320px, 1fr));
    gap: 20px;
}}

.card {{
    border-radius: 12px;
    overflow: hidden;
    box-shadow: 0 4px 14px rgba(0,0,0,0.1);
}}

img {{
    width: 100%;
    height: 250px;
    object-fit: cover;
}}

.conteudo {{
    padding: 12px;
}}

```

```

.filtros {{
  margin: 20px 0;
}}
</style>
</head>

<body>

<h1>Catálogo Digital – Alegrete-RS</h1>
<p>Total de itens: {total}</p>
<p>Atualizado em: {data}</p>

<div class="filtros">
  <input id="busca" placeholder="Buscar...">
  <select id="categoria">
    <option>Todas</option>
    {options}
  </select>
</div>

<div class="grid">
  {".join(cards)}
</div>

<script>
const busca = document.getElementById("busca");
const categoria = document.getElementById("categoria");

function filtrar() {{
  document.querySelectorAll(".card").forEach(card => {{
    const texto = card.innerText.toLowerCase();
    const okBusca = texto.includes(busca.value.toLowerCase());
    const okCat = categoria.value === "Todas" || card.dataset.cat === categoria.value;

    card.style.display = (okBusca && okCat) ? "block" : "none";
  }});
}}

busca.addEventListener("input", filtrar);
categoria.addEventListener("change", filtrar);
</script>

</body>
</html>
""""

# =====

```

```

# EXECUÇÃO
# =====

def executar():
    df = carregar_dados()

    resultado = []

    with requests.Session() as sessao:
        for i, row in df.iterrows():
            print(f"Processando {i+1}/{len(df)}")

            item = row.to_dict()
            item["arquivo"] = baixar_imagem(sessao, item["imagem"], i)

            resultado.append(item)

    cards = [criar_card(i) for i in resultado]

    categorias = sorted(Counter(i["categoria"] for i in resultado))

    Path(CFG.html).write_text(
        gerar_html(cards, categorias, len(resultado)),
        encoding="utf-8"
    )

    Path(CFG.json_file).write_text(
        json.dumps(resultado, indent=2, ensure_ascii=False),
        encoding="utf-8"
    )

    Path(CFG.relatorio).write_text(
        f""PROJETO CONCLUÍDO

Itens: {len(resultado)}
Categorias: {len(categorias)}
Data: {datetime.now()}
""",
        encoding="utf-8"
    )

    print("\nExecução finalizada com sucesso.")

if __name__ == "__main__":
    executar()

```

(iv) ESTRUTURA DO REPOSITÓRIO (GITHUB)

projeto-alegrete/

```
|
|— galeria.py
|— fotos_alegrete.csv
|— portal_alegrete.html
|— catalogo.json
|— relatorio_execucao.txt
|— requirements.txt
|— README.md
|— imagens/
```

v) README.md (RESUMO)

Catálogo Digital – Alegrete-RS

Projeto desenvolvido em Python com apoio de IA para geração automática de catálogo digital e galeria de imagens sobre o município de Alegrete-RS.

Funcionalidades

- leitura de CSV
- download de imagens
- geração de HTML interativo
- exportação JSON
- organização automática de conteúdo

Execução

```
pip install -r requirements.txt
python galeria.py
```

Resultado

Geração automática de um portal digital navegável com conteúdos sobre Alegrete-RS.